

Mixture-of-Density DMD and TDM

A strict derivation, practical surrogates, and a Flow-Factory implementation blueprint

Jayce-Ping

2026-08-03

Abstract

This tutorial proposes two new algorithms, *Mixture-of-Density DMD* (MoD-DMD) and *Mixture-of-Density TDM* (MoD-TDM). At outer iteration k , freeze the old student θ_k and form the probability target

$$q_{k,\tau} = (1 - \alpha)\rho_{\text{old},\tau} + \alpha\rho_{T,\tau}.$$

MoD-DMD minimizes a Gaussian-smoothed reverse KL at the endpoint; MoD-TDM applies the same principle to several ODE-time marginals. The target score is the score of the *old/teacher density mixture*, not the teacher score alone. It can be learned without evaluating either source density by choosing the old or teacher source once per trajectory, adding an independent auxiliary Gaussian corruption, and training a target-mixture denoiser.

The derivation keeps two time variables separate: τ is deterministic source-ODE time, whereas s is auxiliary denoising-score-matching noise time. We prove that smoothing preserves mixtures, derive the smoothed responsibility, score, Tweedie denoiser, and rectified-flow auxiliary-velocity identities with constants and signs explicit, and obtain the exact pathwise reverse-KL gradient

$$\mathbb{E}[\bar{\lambda}(s) a_s J_\theta^\top (\mathbf{s}_{\text{fake}} - \mathbf{s}_q)].$$

Here $\bar{\lambda} = \lambda/r$ includes the importance correction for the sampled auxiliary-time density r . We then derive the first-inner-step responsibility gate, both a multi-marginal extension and a paper-style segment-conditioned TDM extension, continuous and Euler sampler sensitivities, random-time estimators, deterministic path-law singularity, outer-loop contraction, fitting-error bounds, and score-lag bias.

These algorithms are **proposals, not current Flow-Factory features**. Current XDMD, XTDM, and XOPD-DM use the frozen teacher as the real score. They do not construct the frozen old/teacher mixture target. The final sections map the exact gap and give a staged implementation blueprint that preserves detach, scheduler, mixed-precision, and distributed weight-swap invariants.

Contents

1	Executive summary and learning goals	4
1.1	The proposal in one page	4
1.2	Learning goals	4
1.3	Four probability layers and two clocks	5
2	Frozen A4 outer target	5
2.1	Old, current, and teacher are three roles	5
2.2	Branch once per source trajectory	5
3	Two times: source ODE tau and auxiliary noise s	6
3.1	General Gaussian corruption	6
3.2	Rectified-flow special case	6

4	Smoothing, responsibilities, scores, and denoisers	6
4.1	Gaussian smoothing preserves mixtures	6
4.2	Smoothed responsibility and mixture-score identity	7
4.3	Tweedie’s identity with all constants	7
4.4	RF auxiliary velocity: sign check	7
5	Three routes to the frozen target-mixture score	8
5.1	Route A: branch-sampled target score/FM loss (preferred)	8
5.2	Route B: responsibility classifier plus source scores	8
5.3	Route C: mixture-versus-current density ratio	8
5.4	Why source ODE velocity is not a marginal score	8
6	Endpoint MoD-DMD	9
6.1	Smoothed reverse-KL objective	9
6.2	Exact pathwise reverse-KL gradient	9
6.3	Denoiser form and timestep weights	10
6.4	Online fake loss and detach boundaries	10
6.5	Event sum, event mean, self-normalization, and Pseudo-Huber	10
7	The first inner step is responsibility gated	11
8	MoD-TDM: multi-time and segment-conditioned matching	11
8.1	TDM-inspired multi-marginal objective	11
8.2	Direct extension of the paper-style segment objective	12
8.3	Continuous ODE sensitivity	13
8.4	Euler sensitivity and scheduler orientation	13
8.5	Exact BPTT versus current truncated semi-gradient	13
9	Why practical TDM is not deterministic path-space KL	13
10	Proximal and contraction interpretation	15
10.1	Ideal outer recursion	15
10.2	Fitting-error bound and Pinsker	15
11	Score errors, lag, and source choices	16
11.1	Target lag versus fake lag	16
11.2	Classifier-error amplification	16
11.3	Rollout-old is not the fixed pretrained base	16
12	Implementation blueprint for Flow-Factory	16
12.1	Five network/source roles	16
12.2	Conceptual pseudocode	16
12.3	Endpoint and TDM data flow	18
12.4	Configuration concepts, not invented accepted fields	18
12.5	Diagnostics	18
12.6	Distributed and mixed-precision invariants	19
12.7	Exact mapping to current files and the gap	20
13	Compact comparison	21
14	Limitations and recommended implementation order	21
14.1	Limitations	21
14.2	Recommended implementation order	21

15 Internal documents and final checklist	22
16 Conclusion	22

1 Executive summary and learning goals

1.1 The proposal in one page

Fix a condition $C = c$ and suppress c below; every distribution, score, and expectation remains conditional on it. At outer iteration k :

1. freeze the rollout-old source at θ_k and keep the pretrained teacher frozen;
2. define $q_{k,\tau} = (1 - \alpha)\rho_{\text{old},\tau} + \alpha\rho_{T,\tau}$;
3. choose the source branch once per source trajectory, producing exact samples from $q_{k,\tau}$ at every queried τ ;
4. train a target-mixture score on Gaussian-corrupted source states and freeze it for the generator inner loop;
5. train an online fake score on detached live-student states;
6. update the student generator with the detached score difference $s_{\text{fake}} - s_q$, backpropagated through the generated state;
7. refresh old $\leftarrow$ current only at the next outer boundary.

MoD-DMD uses $\tau = 1$. The TDM extension has two precise readings developed below: a TDM-inspired multi-marginal objective with an independent auxiliary channel, and a direct segment-conditioned extension of the paper objective. Neither is ordinary teacher DMD: the target score is an old/teacher mixture score, not a teacher-only score.

Status boundary

The mathematics below specifies a future algorithm. Current XDMD/XTDM supply the teacher as **real**; current XOPD **marginal.cfm** implements A4 marginal-mixture source-velocity regression. No current path combines the *proposed* frozen mixture target $q_{k,\tau,s}$ —including an old freeze boundary and target-score phase—with DMD/TDM’s online fake-score reverse-KL update.

1.2 Learning goals

After reading this note, the reader should be able to:

1. distinguish kernels, ODE fields and flows, one-time marginals, and path laws;
2. keep ODE time τ separate from auxiliary DSM time s ;
3. derive the mixture score and rectified-flow auxiliary target without sign mistakes;
4. identify three valid target-score routes and reject source ODE velocity as a generic score;
5. derive the exact MoD-DMD and MoD-TDM pathwise gradients and their denoiser forms;
6. explain why the first inner force is responsibility gated rather than α times ordinary teacher DMD;
7. distinguish exact sampler BPTT from the current truncated one-step semi-gradient;
8. explain why deterministic path-space KL is generically singular while smoothed marginal KL is practical;
9. state exactly what current XOPDTrainer, XDMDTrainer, XTDMTrainer, and XOPDDMTrainer do and what a future implementation must add.

1.3 Four probability layers and two clocks

Figure 1 separates objects that are often conflated.

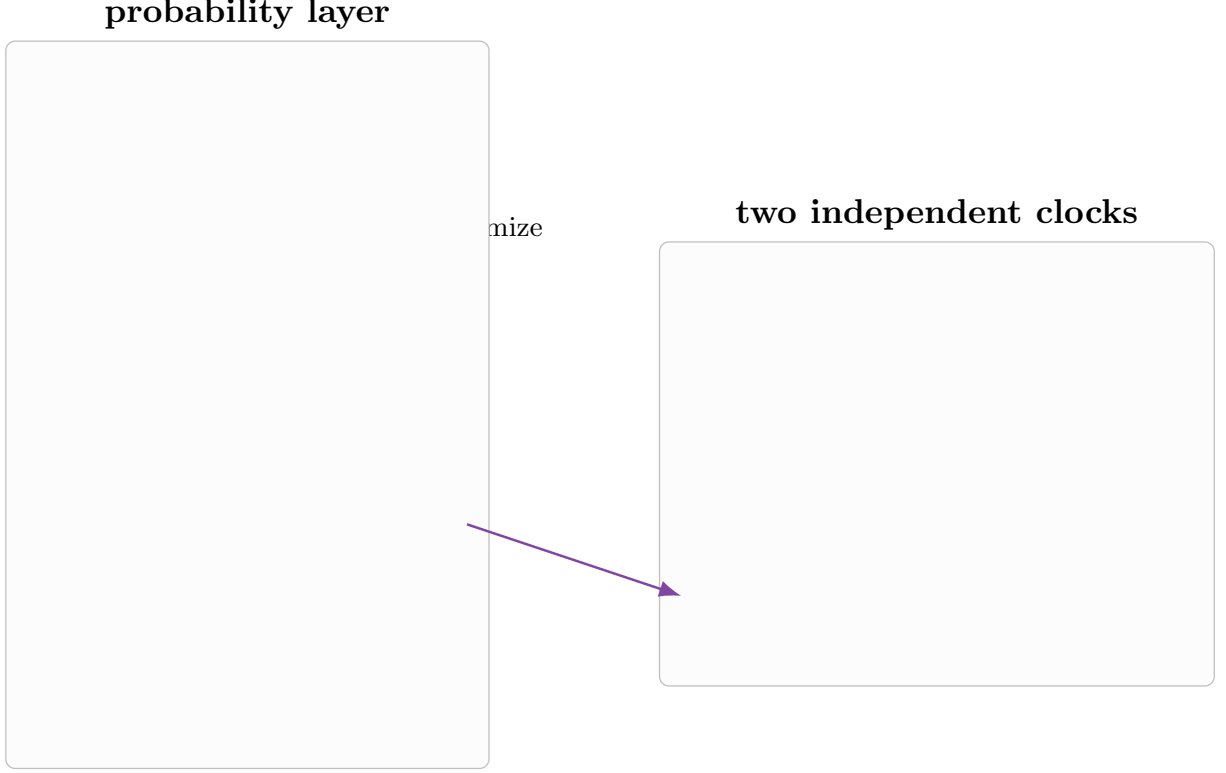


Figure 1: The algorithm matches Gaussian-smoothed one-time marginals. The auxiliary corruption clock s is not the source ODE clock τ , and a collection of matched marginals is not a deterministic path-law KL.

2 Frozen A4 outer target

2.1 Old, current, and teacher are three roles

Let θ_k be the student checkpoint at the start of outer iteration k . The rollout-old source is frozen:

$$\rho_{\text{old},\tau} := \rho_{\theta_k,\tau}.$$

The symbol θ denotes the trainable current student during the inner loop; after its first optimizer step it is no longer equal to θ_k . The teacher T is frozen independently. Choose a fixed outer weight $\alpha \in [0, 1]$ and define

$$q_{k,\tau} := (1 - \alpha)\rho_{\text{old},\tau} + \alpha\rho_{T,\tau}. \quad (1)$$

The complete family $\{q_{k,\tau}\}_{\tau \in [0,1]}$ is frozen during all inner updates.

2.2 Branch once per source trajectory

Draw $B \sim \text{Bernoulli}(\alpha)$ once and initial noise $Z \sim \rho_0$. Then

$$X_\tau = \begin{cases} \Phi_{\tau \leftarrow 0}^{\text{old}}(Z), & B = 0, \\ \Phi_{\tau \leftarrow 0}^T(Z), & B = 1. \end{cases} \quad (2)$$

For every fixed τ , $X_\tau \sim q_{k,\tau}$. Keeping B fixed matters: redrawing it at each Euler step creates a switching control and approaches an arithmetic field average, not the mixture of source-trajectory

marginals. The source path is used to sample the frozen target; the learned student remains one deterministic generator and need not reproduce the random source branch as a path law.

Mathematical old versus the shipped A4 phase boundary

The equations require old to remain fixed while the target score and generator inner loop are fit. Current `marginal_cfm` does not materialize a separately named θ_k copy: its $B = 0$ branch calls the current student during the rollout phase, before optimization, and then reuses cached source states and velocities. A future target-score phase may preserve that strict rollout-then-optimize boundary, or create an explicit snapshot if target fitting overlaps student updates. Reusing A4 routing alone is therefore insufficient; the freeze boundary must also be explicit.

3 Two times: source ODE tau and auxiliary noise s

3.1 General Gaussian corruption

After obtaining a source or student state X_τ , draw $\varepsilon \sim \mathcal{N}(0, I)$ independently and set

$$Y = a_s X_\tau + b_s \varepsilon, \quad b_s > 0. \quad (3)$$

The scalar s indexes an auxiliary Gaussian channel. It need not coincide with any scheduler time used to produce X_τ . We write $\rho_{j,\tau,s}$ for the density of Y when $X_\tau \sim \rho_{j,\tau}$.

3.2 Rectified-flow special case

The RF corruption used in current Flow-Factory DMD helpers is

$$a_s = 1 - s, \quad b_s = s, \quad Y = (1 - s)X_\tau + s\varepsilon, \quad s \in (0, 1]. \quad (4)$$

Current scheduler code may represent s as a decreasing noise fraction $\sigma = t/1000$. This is a numerical orientation convention. It does not identify s with the source time τ .

Two-time warning

The auxiliary DSM velocity $\mathbb{E}[\varepsilon - X_\tau \mid Y]$ describes the conditional target of the artificial line $Y_s = (1 - s)X_\tau + s\varepsilon$. It is *not* the source ODE velocity $v_j(\tau, X_\tau)$ that generated the clean state.

4 Smoothing, responsibilities, scores, and denoisers

4.1 Gaussian smoothing preserves mixtures

Let

$$\varphi_{b_s}(u) = (2\pi b_s^2)^{-d/2} \exp\left(-\frac{\|u\|^2}{2b_s^2}\right).$$

For source $j \in \{\text{old}, T\}$,

$$\rho_{j,\tau,s}(y) = \int_{\mathbb{R}^d} \varphi_{b_s}(y - a_s x) \rho_{j,\tau}(x) dx. \quad (5)$$

Proposition 4.1 (Linearity of smoothing). *The smoothed A4 target is*

$$q_{k,\tau,s}(y) = (1 - \alpha) \rho_{\text{old},\tau,s}(y) + \alpha \rho_{T,\tau,s}(y). \quad (6)$$

Proof. Insert (1) into the convolution integral and use linearity:

$$\int \varphi_{b_s}(y - a_s x) [(1 - \alpha)\rho_{\text{old},\tau}(x) + \alpha\rho_{T,\tau}(x)] dx = (1 - \alpha)\rho_{\text{old},\tau,s}(y) + \alpha\rho_{T,\tau,s}(y).$$

□

4.2 Smoothed responsibility and mixture-score identity

Where $q_{k,\tau,s}(y) > 0$, define exactly

$$\boxed{\gamma_{k,\tau,s}(y) := \frac{\alpha\rho_{T,\tau,s}(y)}{q_{k,\tau,s}(y)}}. \quad (7)$$

This is the posterior probability $\Pr(B = 1 \mid Y = y, \tau, s)$. It is neither A4's unsmoothed responsibility $\alpha\rho_{T,\tau}/q_{k,\tau}$ nor an SDE transition gate.

Proposition 4.2 (Mixture score). *Writing $\mathbf{s}_j = \nabla_y \log \rho_{j,\tau,s}$ and $\mathbf{s}_q = \nabla_y \log q_{k,\tau,s}$,*

$$\boxed{\mathbf{s}_q(y) = (1 - \gamma_{k,\tau,s}(y))\mathbf{s}_{\text{old}}(y) + \gamma_{k,\tau,s}(y)\mathbf{s}_T(y)}. \quad (8)$$

Proof. Differentiate (6), use $\nabla \rho_j = \rho_j \mathbf{s}_j$, and divide by $q_{k,\tau,s}$. □

4.3 Tweedie's identity with all constants

Proposition 4.3 (General Gaussian score and denoiser [6]). *For $Y = a_s X + b_s \varepsilon$ with $b_s > 0$,*

$$\boxed{\nabla_y \log p_s(y) = \frac{a_s \mathbb{E}[X \mid Y = y] - y}{b_s^2}}. \quad (9)$$

If $a_s \neq 0$, the posterior-mean clean estimate is

$$\boxed{\hat{x}_p(y, s) := \mathbb{E}[X \mid Y = y] = \frac{y + b_s^2 \mathbf{s}_p(y, s)}{a_s}}. \quad (10)$$

Proof. Differentiate under the integral:

$$\begin{aligned} \nabla_y p_s(y) &= \int -\frac{y - a_s x}{b_s^2} \varphi_{b_s}(y - a_s x) p(x) dx \\ &= \frac{a_s \mathbb{E}[X \mid Y = y] - y}{b_s^2} p_s(y). \end{aligned}$$

Division by $p_s(y)$ proves (9); rearranging proves (10). □

4.4 RF auxiliary velocity: sign check

Under (4), define

$$d_p(y, s) := \mathbb{E}[\varepsilon - X \mid Y = y]. \quad (11)$$

Because $Y = X + s(\varepsilon - X)$,

$$\boxed{\hat{x}_p(y, s) = \mathbb{E}[X \mid Y = y] = y - s d_p(y, s)}. \quad (12)$$

Substituting $a_s = 1 - s$, $b_s = s$ and (12) into (9) gives

$$\begin{aligned} \mathbf{s}_p(y, s) &= \frac{(1 - s)(y - s d_p) - y}{s^2} \\ &= \boxed{-\frac{y + (1 - s)d_p(y, s)}{s}}. \end{aligned} \quad (13)$$

Both the minus sign and the factor $(1 - s)$ are essential. A denoising regression

$$\mathbb{E} \|d_\eta(Y, \tau, s) - (\varepsilon - X_\tau)\|^2$$

has population minimizer d_p ; it does not estimate $v(\tau, \cdot)$.

5 Three routes to the frozen target-mixture score

5.1 Route A: branch-sampled target score/FM loss (preferred)

Sample B once per source trajectory, query X_τ on that trajectory, corrupt it with fresh (s, ε) , and regress a target network onto the auxiliary label. Marginalizing B produces $X_\tau \sim q_{k,\tau}$; ordinary conditional regression therefore learns the posterior-averaged d_q or equivalent score. No density or classifier is evaluated.

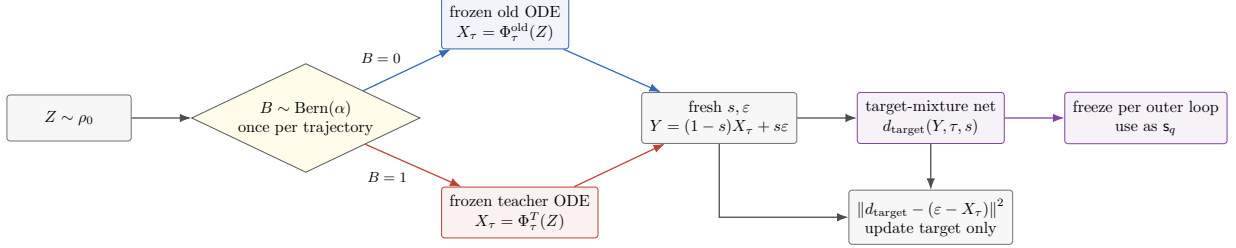


Figure 2: Preferred RF target-score training. The source branch generates the correct target marginal; the auxiliary Gaussian channel supplies a standard DSM/FM label. The resulting target-mixture score is frozen during generator inner updates.

5.2 Route B: responsibility classifier plus source scores

Train a calibrated classifier $r_\xi(y, \tau, s) \approx \Pr(B = 1 \mid y, \tau, s)$ and source score models $\hat{s}_{\text{old}}, \hat{s}_T$. Then use

$$\hat{s}_q = (1 - r_\xi)\hat{s}_{\text{old}} + r_\xi\hat{s}_T.$$

This route is analytically transparent and exposes $\gamma_{k,\tau,s}$ for diagnostics, but it requires three accurate learned objects. When source scores differ greatly, small classifier error is amplified.

5.3 Route C: mixture-versus-current density ratio

Train a discriminator between $Y_q \sim q_{k,\tau,s}$ and $Y_p \sim p_{\theta,\tau,s}$. With equal class priors and optimal logit $\ell^* = \log q_{k,\tau,s} - \log p_{\theta,\tau,s}$,

$$\nabla_y \ell^*(y) = s_q(y) - s_{\text{fake}}(y). \quad (14)$$

This directly estimates the force needed by reverse KL, but it must track the moving current student and can be ill-conditioned when the two densities have little overlap.

5.4 Why source ODE velocity is not a marginal score

The continuity equation

$$\partial_\tau \rho_\tau + \nabla \cdot (\rho_\tau v_\tau) = 0$$

does not determine $\nabla \log \rho_\tau$ from the local value $v_\tau(x)$. The score depends on the global transported density and the flow Jacobian. A probability-flow ODE can be related to a score only after specifying a particular forward diffusion and drift/diffusion coefficients. Therefore neither $v_{\text{old}}(\tau, x)$ nor $v_T(\tau, x)$ may be substituted for the score of the intermediate marginal $\rho_{j,\tau,s}$ without a separate proof.

Listing 1: Wrong: source ODE velocity is not a generic DSM score.

```

1 # X_tau was produced by the source ODE at trajectory time tau.
2 score_q = (1 - alpha) * old.velocity(tau, X_tau) \
3         + alpha * teacher.velocity(tau, X_tau)
4 loss = dmd_surrogate(score_fake - score_q)

```

Listing 2: Correct conceptual target-score sampling.

```

1 # branch is sampled once and held fixed for the complete source trajectory.
2 X_tau = frozen_source_trajectory(branch).state_at(tau)
3 epsilon = torch.randn_like(X_tau)
4 Y = a(s) * X_tau + b(s) * epsilon
5 aux_target = epsilon - X_tau # RF auxiliary velocity label
6 loss_target = mse(target_net(Y, tau, s), aux_target)

```

6 Endpoint MoD-DMD

6.1 Smoothed reverse-KL objective

Let $X = G_\theta(Z)$ be the current endpoint sample and let $p_{\theta,1,s}$ be the density of $Y = a_s X + b_s \varepsilon$. For a nonnegative integrable weight $\lambda(s)$, define

$$\mathcal{L}_{\text{MoD-DMD}}(\theta; q_k) = \int_S \lambda(s) \text{KL}(p_{\theta,1,s} \| q_{k,1,s}) \, ds. \quad (15)$$

In Monte Carlo, draw $S \sim r$ from a density that is positive wherever λ is nonzero and write $\bar{\lambda}(S) := \lambda(S)/r(S)$. The target q_k is held fixed while differentiating. This is the mixture-target counterpart of score-based implicit-distribution matching used by Diff-Instruct and DMD/DMD2 [2, 3, 4].

The displayed G_θ is sampler-agnostic. Current `XMDDTrainer` instantiates it as a DMD2 consistency-style backward simulation: a no-grad predict- x_0 /fresh-re-noise prefix and one differentiable clean prediction, rather than a differentiable ODE endpoint.

6.2 Exact pathwise reverse-KL gradient

Write $J_\theta(Z) = \partial G_\theta(Z)/\partial \theta$ and

$$\mathbf{s}_{\text{fake}}(y, s) = \nabla_y \log p_{\theta,1,s}(y), \quad \mathbf{s}_q(y, s) = \nabla_y \log q_{k,1,s}(y).$$

Theorem 6.1 (Pathwise MoD-DMD gradient). *Under the usual differentiability, integrability, and vanishing-boundary conditions,*

$$\nabla_\theta \mathcal{L}_{\text{MoD-DMD}} = \mathbb{E}_{Z, \varepsilon, s} \left[\bar{\lambda}(s) a_s J_\theta(Z)^\top (\mathbf{s}_{\text{fake}}(Y, s) - \mathbf{s}_q(Y, s)) \right]. \quad (16)$$

Thus gradient descent moves samples in the direction $\mathbf{s}_q - \mathbf{s}_{\text{fake}}$.

Proof. For one s , differentiate the reparameterized expectation of $\log p_{\theta,1,s}(Y) - \log q_{k,1,s}(Y)$. The pathwise derivative through Y contributes $(\mathbf{s}_{\text{fake}} - \mathbf{s}_q)^\top \partial_\theta Y$. The expected explicit parameter derivative of $\log p_{\theta,1,s}$ vanishes by score normalization. Since $\partial_\theta Y = a_s J_\theta$, sampling s with the factor $\bar{\lambda}(s) = \lambda(s)/r(s)$ gives the integral in (16). $\square$

6.3 Denoiser form and timestep weights

From (10),

$$\mathbf{s}_{\text{fake}} - \mathbf{s}_q = \frac{a_s}{b_s^2} (\hat{x}_{\text{fake}} - \hat{x}_q). \quad (17)$$

Therefore

$$\nabla_{\theta} \mathcal{L}_{\text{MoD-DMD}} = \mathbb{E} \left[\bar{\lambda}(s) \frac{a_s^2}{b_s^2} J_{\theta}^{\top} (\hat{x}_{\text{fake}} - \hat{x}_q) \right]. \quad (18)$$

Using the bare clean-estimate residual $\hat{x}_{\text{fake}} - \hat{x}_q$ is legitimate only as a deliberately reweighted surrogate: it replaces the exact effective weight $\lambda(s)a_s^2/b_s^2$ by another timestep weighting.

For RF, (12) gives

$$\hat{x}_{\text{fake}} - \hat{x}_q = -s(d_{\text{fake}} - d_q).$$

Again, a bare auxiliary-velocity residual changes the s weighting unless the compensating factor is included.

6.4 Online fake loss and detach boundaries

The fake network estimates the score of the *live* student distribution. Generate a current endpoint X_{θ} , detach it, add fresh auxiliary noise, and optimize

$$\mathcal{L}_{\text{fake}}(\phi) = \mathbb{E} \|d_{\phi}(Y, s) - (\epsilon - \text{sg } X_{\theta})\|^2. \quad (19)$$

During generator backward, both target and fake predictions are detached. Only $X_{\theta} = G_{\theta}(Z)$ retains a generator graph.

Listing 3: Conceptual endpoint generator update with exact detach roles.

```

1 X = student_generator(noise, condition)           # keeps generator graph
2 s, epsilon = sample_auxiliary_corruption(X.detach())
3 Y_value = a(s) * X.detach() + b(s) * epsilon     # score-query value
4
5 with torch.no_grad():
6     score_q = frozen_target_mixture.score(Y_value, tau=1.0, s=s)
7     score_fake = online_fake.score(Y_value, tau=1.0, s=s)
8     force = importance_weight(s) * a(s) * (score_fake - score_q)
9
10 # d(loss)/dX equals detached force; only X backpropagates.
11 target = (X.float() - force.float()).detach()
12 loss_gen = 0.5 * event_reduce((X.float() - target).square())
13 loss_gen.backward()

```

6.5 Event sum, event mean, self-normalization, and Pseudo-Huber

For event dimension D , the exact Euclidean inner product corresponds to an event sum. An event mean multiplies the complete gradient by $1/D$ at fixed shape, preserving stationary points but changing resolution-dependent optimization scale. It should be documented as a normalization, not as a different KL.

DMD2-style self-normalization divides the detached force by a sample-dependent scale such as $\text{mean}[p_{\text{target}}]$. This generally changes relative weighting across examples and timesteps; it is a stability heuristic, not the exact gradient of (15). Pseudo-Huber similarly rescales large residuals. It can stabilize heavy-tailed forces, but its gradient is not the unmodified reverse-KL force unless the induced scalar multiplier is explicitly included in the intended objective.

Algorithm 1: one outer iteration of proposed MoD-DMD

1. Snapshot θ_k as the frozen rollout-old source.
2. Sample complete old/teacher source trajectories with one Bernoulli branch per trajectory.
3. Train the target-mixture DSM/FM network on (X_1, s, ε) ; then freeze it.
4. Repeatedly train the online fake score on detached live-student endpoints.
5. For each generator update, retain the student sampler graph, query both scores under no-grad, construct the detached force $\bar{\lambda}(s)a_s(\mathbf{s}_{\text{fake}} - \mathbf{s}_q)$, and backpropagate only through the endpoint.
6. At the next outer boundary, evaluate fitting diagnostics and refresh $\text{old} \leftarrow \theta$.

7 The first inner step is responsibility gated

Theorem 7.1 (First-inner-step force). *Suppose the inner loop begins with $p_{\theta, \tau, s} = \rho_{\text{old}, \tau, s}$ under the same Gaussian channel and the score estimates are exact. Then*

$$\mathbf{s}_{\text{fake}} - \mathbf{s}_q = \gamma_{k, \tau, s}(\mathbf{s}_{\text{old}} - \mathbf{s}_T), \quad (20)$$

so the negative-gradient sample force is

$$\boxed{\mathbf{s}_q - \mathbf{s}_{\text{fake}} = \gamma_{k, \tau, s}(\mathbf{s}_T - \mathbf{s}_{\text{old}})}. \quad (21)$$

Proof. At initialization $\mathbf{s}_{\text{fake}} = \mathbf{s}_{\text{old}}$. Substitute (8) and simplify. $\square$

The gate is the *smoothed posterior responsibility* $\gamma_{k, \tau, s}(y)$, not the prior α . Therefore the first step is not “ α times teacher DMD.” Where teacher density is negligible, reverse KL gives little teacher-directed force even if α is moderate. Increasing auxiliary smoothing increases overlap and softens the responsibility boundary, but excessive smoothing removes fine target structure.

Reverse KL is mode seeking: a finite-capacity student may concentrate on a subset of mixture modes rather than representing the full convex mixture. One optimizer step only takes a local parameter-space move under J_θ ; it does not set $p_\theta = q_k$. Exact realization of q_k requires sufficient capacity, score accuracy, sampler sensitivity, and inner optimization.

8 MoD-TDM: multi-time and segment-conditioned matching

8.1 TDM-inspired multi-marginal objective

Choose trajectory times $0 < \tau_1 < \dots < \tau_m \leq 1$ and weights $w_i \geq 0$. Define

$$\mathcal{L}_{\text{MoD-TDM}}(\theta; q_k) = \sum_{i=1}^m w_i \int_{\mathcal{S}} \lambda_i(s) \text{KL}(p_{\theta, \tau_i, s} \| q_{k, \tau_i, s}) \text{d}s. \quad (22)$$

Endpoint MoD-DMD is the special case $m = 1$, $\tau_1 = 1$. The multi-time construction is motivated by trajectory distribution matching [5], but its real target here is the frozen old/teacher mixture rather than the teacher alone.

If $I \sim \pi$ with $\pi_i > 0$, then

$$\mathbb{E}_I \left[\frac{w_I}{\pi_I} \nabla_\theta \int \lambda_I(s) \text{KL}(p_{\theta, \tau_I, s} \| q_{k, \tau_I, s}) \text{d}s \right] = \nabla_\theta \mathcal{L}_{\text{MoD-TDM}}. \quad (23)$$

Uniform random segment selection is unbiased for a uniform sum only when the corresponding m factor is included; omitting it merely rescales the whole objective if all other weights are equal, but nonuniform sampling requires w_i/π_i . Likewise, sampling $S \sim r_i$ inside the chosen term requires $\lambda_i(S)/r_i(S)$.

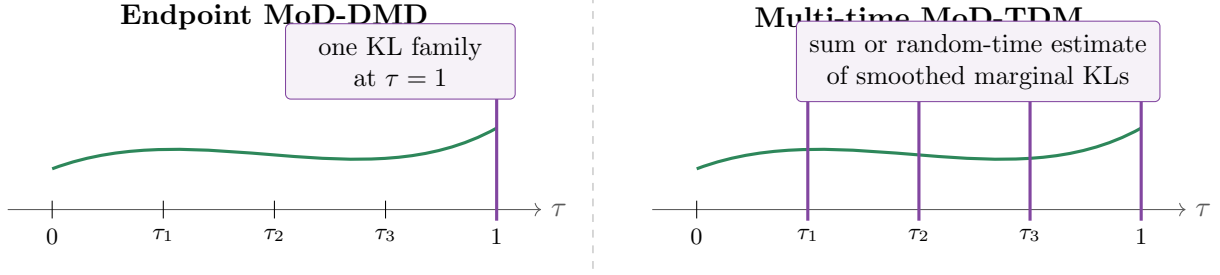


Figure 3: Endpoint DMD constrains one generated marginal. TDM distributes score-difference forces across trajectory times. The vertical bars denote auxiliary-smoothed marginal KL families, not deterministic path-space KL.

8.2 Direct extension of the paper-style segment objective

Equation (22) is best described as *TDM-inspired multi-marginal DMD*: it first selects a clean student/source marginal at τ_i and then applies an independently indexed auxiliary channel K_s to both sides. Paper-style TDM instead anchors at a segment boundary and uses the native forward kernel over a non-overlapping segment. Let I_i be the i th segment and let $K_{u|\tau_i}$ denote that kernel for $u \in I_i$. Define

$$p_{\theta,u}^{(i)} := (K_{u|\tau_i})_{\#} p_{\theta,\tau_i}, \quad q_{k,u} := (1 - \alpha) \rho_{\text{old},u} + \alpha \rho_{T,u}. \quad (24)$$

The paper-faithful mixture-target replacement is

$$\mathcal{L}_{\text{native-seg-MoD-TDM}} := \sum_i \int_{u \in I_i} \lambda_i(u) \text{KL}(p_{\theta,u}^{(i)} \| q_{k,u}) du. \quad (25)$$

Here the target is the native-time source mixture at u ; branch-fixed old/teacher trajectories provide direct samples from it. Samples alone do not provide an exact score label. The native $\mathbf{s}_q(\cdot, u)$ therefore requires either valid source native-time scores plus a responsibility estimator, an explicit score-matching method, or another justified estimator. Applying an additional DSM corruption is often practical, but it changes the target to a further-smoothed native segment law and must be written as such.

A second, valid but different construction symmetrically applies the segment kernel to the anchor target:

$$\begin{aligned} \tilde{q}_{k,u}^{(i)} &:= (K_{u|\tau_i})_{\#} q_{k,\tau_i} \\ &= (1 - \alpha) (K_{u|\tau_i})_{\#} \rho_{\text{old},\tau_i} + \alpha (K_{u|\tau_i})_{\#} \rho_{T,\tau_i}. \end{aligned} \quad (26)$$

Linearity preserves the mixture, so branch sampling also trains this score. The native target $q_{k,u}$ and symmetric target $\tilde{q}_{k,u}^{(i)}$ coincide only when each source is consistent with that same kernel,

$$(K_{u|\tau_i})_{\#} \rho_{j,\tau_i} = \rho_{j,u}, \quad j \in \{\text{old}, T\},$$

and should otherwise be named as different objectives.

For a reparameterized segment kernel $Y_u = T_{u|\tau_i}(X_{\tau_i}, \varepsilon)$, the native-target gradient is

$$\nabla_{\theta} \mathcal{L}_{\text{native-seg}} = \sum_i \int_{I_i} \lambda_i(u) \mathbb{E} \left[S_{\theta}(\tau_i)^{\top} D_x T_{u|\tau_i}^{\top} (\mathbf{s}_{\text{fake}}^{(i)}(Y_u, u) - \mathbf{s}_q(Y_u, u)) \right] du. \quad (27)$$

If $I \sim \pi$ and $U \sim r_I$, its Monte Carlo multiplier is $\lambda_I(U)/[\pi_I r_I(U)]$. For an affine kernel $Y_u = a_{i,u}X_{\tau_i} + b_{i,u}\varepsilon$, the Jacobian factor reduces to $a_{i,u}S_\theta(\tau_i)^\top$.

Thus (22) and (25) are related but not identical. They coincide only after identifying K_s with $K_{u|\tau_i}$, matching the sampling measures, and establishing the required target identities. Current Flow-Factory **XTDMTrainer** follows a practical segment-banded surrogate: it treats the selected post-Euler state as a clean anchor, re-noises it in the segment’s non-overlapping scheduler band, and queries teacher/fake denoisers. That engineering rule is not automatically the independent two-clock objective (22).

8.3 Continuous ODE sensitivity

For noise-to-data orientation,

$$\frac{dX_\theta(\tau)}{d\tau} = v_\theta(\tau, X_\theta(\tau)), \quad X_\theta(0) = Z. \quad (28)$$

Let $S_\theta(\tau) = \partial_\theta X_\theta(\tau)$. Differentiating gives

$$\frac{dS_\theta}{d\tau} = \partial_x v_\theta(\tau, X_\theta)S_\theta + \partial_\theta v_\theta(\tau, X_\theta), \quad S_\theta(0) = 0. \quad (29)$$

The exact marginal gradient at τ_i contains $a_s S_\theta(\tau_i)^\top (\mathbf{s}_{\text{fake}} - \mathbf{s}_q)$.

8.4 Euler sensitivity and scheduler orientation

For an increasing mathematical grid with $h_i = \tau_{i+1} - \tau_i > 0$,

$$X_{i+1} = X_i + h_i v_\theta(\tau_i, X_i), \quad (30)$$

$$S_{i+1} = (I + h_i \partial_x v_\theta(\tau_i, X_i))S_i + h_i \partial_\theta v_\theta(\tau_i, X_i). \quad (31)$$

FLUX-style scheduler code often uses decreasing σ_i . Its actual update is

$$X_{i+1} = X_i + (\sigma_{i+1} - \sigma_i) v_\theta(\sigma_i, X_i), \quad \sigma_{i+1} - \sigma_i < 0. \quad (32)$$

The sensitivity must use this signed increment. Silently replacing it by $|\sigma_{i+1} - \sigma_i|$ reverses the generator Jacobian contribution.

8.5 Exact BPTT versus current truncated semi-gradient

Exact BPTT retains all terms in (31). The current XTDM v1 pattern rolls out the prefix under `no_grad` and keeps only one selected Euler step differentiable. If X_i is detached, then

$$S_{i+1}^{\text{trunc}} = (\sigma_{i+1} - \sigma_i) \partial_\theta v_\theta(\sigma_i, \text{sg } X_i). \quad (33)$$

It omits the sensitivity of all earlier steps and the propagation term $(I + \Delta\sigma_i \partial_x v_i)S_i$. It is a memory-efficient truncated *semi-gradient*, not the exact gradient of the full multi-time objective.

9 Why practical TDM is not deterministic path-space KL

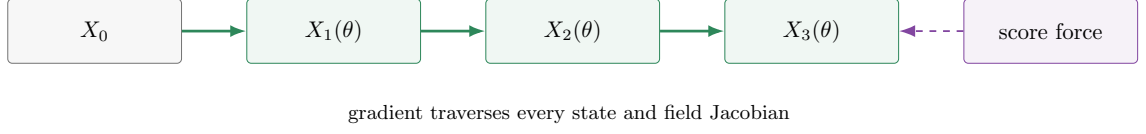
On a grid $\tau_0, \dots, \tau_m$, a deterministic ODE maps one initial state $z \in \mathbb{R}^d$ to

$$F_\theta(z) = (z, \Phi_{\tau_1}^\theta(z), \dots, \Phi_{\tau_m}^\theta(z)) \in \mathbb{R}^{d(m+1)}.$$

The path law is supported on the d -dimensional graph $\mathcal{G}_\theta = \{F_\theta(z) : z \in \mathbb{R}^d\}$.

Proposition 9.1 (Generic path-law singularity). *For two deterministic flows, let $E = \{z : F_1(z) = F_2(z)\}$. If $\rho_0(E) = 0$, then their discretized path laws are mutually singular. Consequently, either directed path-space KL is infinite.*

Exact BPTT



Current v1 truncated step

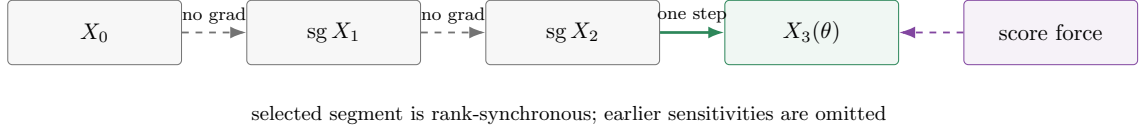


Figure 4: Exact sampler sensitivity versus the current no-grad-prefix plus one differentiable step approximation. The score force is detached in both cases; the difference is the state Jacobian retained by the generator.

Proof. The first law assigns probability one to $\mathcal{G}_1$. Because the first path coordinate is the initial state z , a point from the second graph lies in $\mathcal{G}_1$ only when $F_1(z) = F_2(z)$. Hence the second law assigns $\mathcal{G}_1$ probability $\rho_0(E) = 0$. Interchanging the two laws proves mutual singularity, which rules out absolute continuity required for finite KL. $\square$

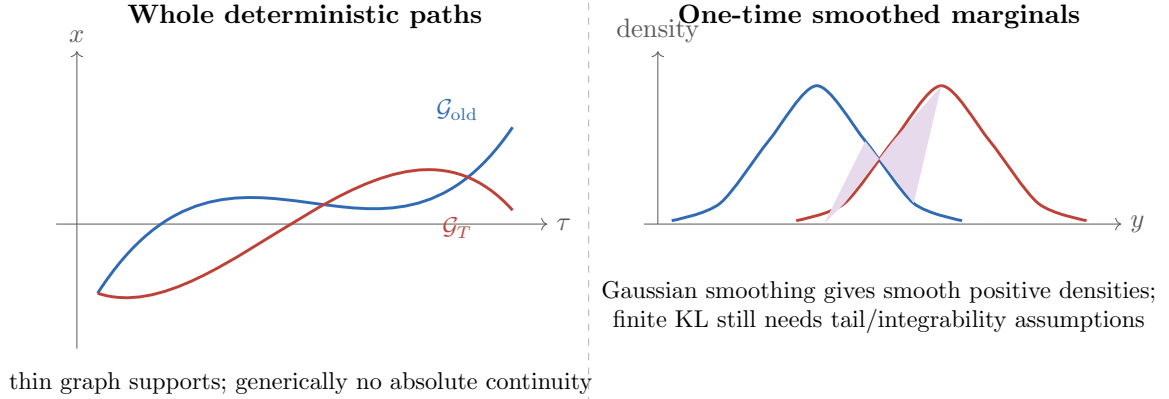


Figure 5: Distinct deterministic path graphs are generically singular, even when their one-time marginals overlap. MoD-TDM therefore matches Gaussian-smoothed marginals, not deterministic path laws.

Branch-fixed source trajectories remain the correct way to sample each $q_{k,\tau}$ jointly and efficiently. Matching all one-time marginals does not imply that the student's path law equals the old/teacher path mixture: many processes share the same marginal family, and the learned student is one deterministic flow.

10 Proximal and contraction interpretation

10.1 Ideal outer recursion

Suppose each outer solve is exact at a fixed τ :

$$\rho_{k+1,\tau} = (1 - \alpha_k)\rho_{k,\tau} + \alpha_k\rho_{T,\tau}. \quad (34)$$

Assume $\alpha_k \in [0, 1]$, the displayed KL terms are finite, and both laws have finite p th moments for $p \geq 1$. Let $A_K = \prod_{j=0}^{K-1} (1 - \alpha_j)$. Induction gives

$$\rho_{K,\tau} = A_K\rho_{0,\tau} + (1 - A_K)\rho_{T,\tau}. \quad (35)$$

For $\text{TV}(\mu, \nu) = \frac{1}{2} \|\mu - \nu\|_1$,

$$\text{TV}(\rho_{k+1,\tau}, \rho_{T,\tau}) = (1 - \alpha_k)\text{TV}(\rho_{k,\tau}, \rho_{T,\tau}), \quad (36)$$

$$\text{KL}(\rho_{k+1,\tau} \| \rho_{T,\tau}) \leq (1 - \alpha_k)\text{KL}(\rho_{k,\tau} \| \rho_{T,\tau}), \quad (37)$$

$$\mathcal{W}_p^p(\rho_{k+1,\tau}, \rho_{T,\tau}) \leq (1 - \alpha_k)\mathcal{W}_p^p(\rho_{k,\tau}, \rho_{T,\tau}). \quad (38)$$

The KL inequality is convexity in the first argument. The Wasserstein bound couples the teacher component to itself and transports only old mass.

10.2 Fitting-error bound and Pinsker

Let the actual fitted law $\hat{\rho}_{k+1,\tau}$ satisfy

$$\varepsilon_{k,\tau} := \text{TV}(\hat{\rho}_{k+1,\tau}, q_{k,\tau}).$$

Then

$$\text{TV}(\hat{\rho}_{k+1,\tau}, \rho_{T,\tau}) \leq \varepsilon_{k,\tau} + (1 - \alpha_k)\text{TV}(\rho_{k,\tau}, \rho_{T,\tau}). \quad (39)$$

Iterating yields

$$D_K \leq A_K D_0 + \sum_{j=0}^{K-1} \left[\prod_{\ell=j+1}^{K-1} (1 - \alpha_\ell) \right] \varepsilon_j, \quad (40)$$

where $D_k = \text{TV}(\rho_{k,\tau}, \rho_{T,\tau})$. If one separately certifies the *unsmoothed* bound $\text{KL}(\hat{\rho}_{k+1,\tau} \| q_{k,\tau}) \leq \delta_{k,\tau}$, Pinsker gives

$$\varepsilon_{k,\tau} \leq \sqrt{\delta_{k,\tau}/2}. \quad (41)$$

This unsmoothed certificate does not follow from a small value of the proposed Gaussian-smoothed objective: data processing gives

$$\text{KL}((K_s)_\# p \| (K_s)_\# q) \leq \text{KL}(p \| q),$$

which is the opposite direction. At a fixed smoothing level one obtains only

$$\text{TV}(\hat{\rho}_{k+1,\tau,s}, q_{k,\tau,s}) \leq \sqrt{\text{KL}(\hat{\rho}_{k+1,\tau,s} \| q_{k,\tau,s})/2}.$$

Exact equality after an informative Gaussian convolution with $a_s \neq 0$ identifies the unsmoothed law, but approximate deconvolution is unstable without additional regularity.

What reverse KL does and does not inherit

The ideal mixture recursion has exact TV contraction and the stated KL/Wasserstein upper bounds. Smoothed reverse-KL training inherits unsmoothed outer contraction only with a separate unsmoothed fitting guarantee or additional stability assumptions. With finite capacity, reverse KL may mode-select inside q_k ; then fitting error can dominate the ideal contraction.

11 Score errors, lag, and source choices

11.1 Target lag versus fake lag

The target-mixture score should be trained from the frozen old/teacher source and then held fixed for one outer inner loop. Its lag is an *outer-target approximation* error. The fake score tracks the changing live student and therefore has *online lag*. Mixing these update semantics destroys the frozen-target interpretation.

Let

$$\hat{\mathbf{s}}_q = \mathbf{s}_q + e_q, \quad \hat{\mathbf{s}}_{\text{fake}} = \mathbf{s}_{\text{fake}} + e_{\text{fake}}.$$

The generator-gradient bias obeys

$$\|\text{Bias}\| \leq \mathbb{E}[\bar{\lambda}(s)|a_s| \|J_\theta\| (\|e_{\text{fake}}\| + \|e_q\|)]. \quad (42)$$

This directly motivates target holdout DSM loss, fake freshness ratios, and score-difference norm diagnostics.

11.2 Classifier-error amplification

For route B, let $\hat{\gamma} = \gamma + \delta_\gamma$ and source score errors be e_{old}, e_T . Then

$$\hat{\mathbf{s}}_q - \mathbf{s}_q = \delta_\gamma(\mathbf{s}_T - \mathbf{s}_{\text{old}}) + (1 - \hat{\gamma})e_{\text{old}} + \hat{\gamma}e_T. \quad (43)$$

Even a small δ_γ is dangerous where the source scores are far apart. Low-overlap regions also produce extreme class probabilities and poor calibration.

11.3 Rollout-old is not the fixed pretrained base

The proximal source must be θ_k , refreshed at the outer boundary. Replacing it by a fixed pretrained base ρ_B changes the target to $(1 - \alpha)\rho_B + \alpha\rho_T$ every time and removes recursion (34). A deliberate three-way target is possible:

$$q_{k,\tau} = (1 - \alpha - \beta)\rho_{\text{old},\tau} + \alpha\rho_{T,\tau} + \beta\rho_{B,\tau}, \quad \alpha, \beta \geq 0, \quad \alpha + \beta \leq 1. \quad (44)$$

It should be named and analyzed as a three-source anchor, not silently implemented by confusing old with base.

12 Implementation blueprint for Flow-Factory

12.1 Five network/source roles

12.2 Conceptual pseudocode

Listing 4: Proposed outer/inner structure; names are conceptual, not accepted config/API fields.

```

1 old_source.load_snapshot(student.state_dict()) # conceptual API
2
3 # Phase A: fit q_k score from branch-fixed source trajectories.
4 for source_batch in target_score_batches:
5     branch = collective_safe_bernoulli(alpha, shape=source_batch.shape)
6     trajectory = rollout_selected_frozen_source_once(
7         old_source=old_source,
8         teacher_source=teacher_source,
9         branch=branch,
10    )
11    tau_q = sample_target_trajectory_time() # 1.0 for endpoint MoD-DMD

```

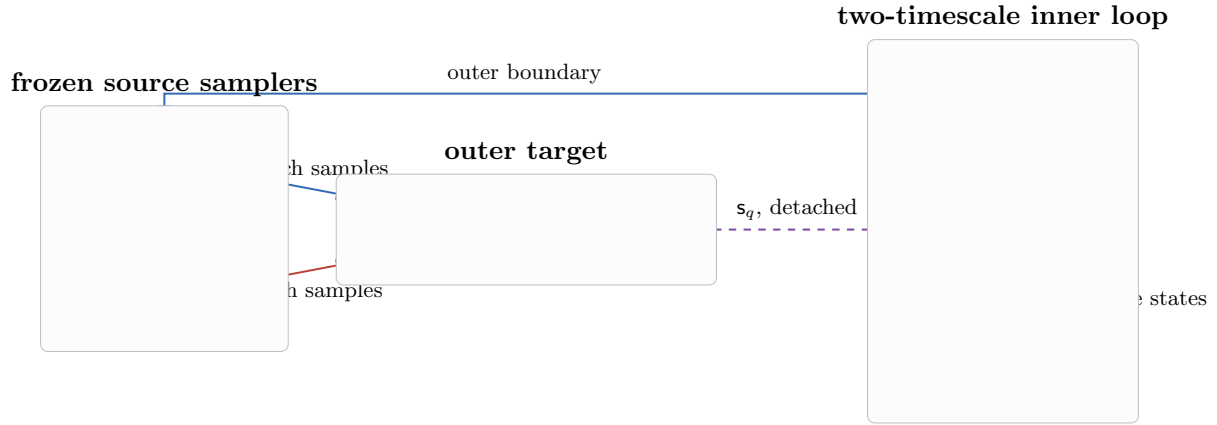



Figure 6: Proposed roles and update loops. Old and fake are distinct: old defines the frozen proximal target; fake estimates the moving current student’s score.

```

12 X_q = trajectory.state_at(tau_q).detach()
13 s_q, eps_q = sample_auxiliary_corruption(X_q)
14 Y_q = a(s_q) * X_q + b(s_q) * eps_q
15 loss_target = mse(target_score.aux_velocity(Y_q, tau_q, s_q),
16                  eps_q - X_q)
17 update_target_score_only(loss_target)
18 target_score.freeze()
19
20 # Phase B: two-timescale current-student updates.
21 for inner_step in inner_steps:
22     X_live, tau_live = rollout_student_with_requested_jacobian()
23
24     # Fake trains on detached live states.
25     s_f, eps_f = sample_auxiliary_corruption(X_live.detach())
26     Y_f = a(s_f) * X_live.detach() + b(s_f) * eps_f
27     loss_fake = mse(fake_score.aux_velocity(Y_f, tau_live, s_f),
28                   eps_f - X_live.detach())
29     update_fake_only(loss_fake)
30
31     # Generator sees detached scores, but X_live keeps its sampler graph.
32     s_g, eps_g = sample_auxiliary_corruption(X_live.detach())
33     with torch.no_grad():
34         Y_value = a(s_g) * X_live.detach() + b(s_g) * eps_g
35         score_q = target_score.score(Y_value, tau_live, s_g)
36         score_fake = fake_score.score(Y_value, tau_live, s_g)
37         force = importance_weight(tau_live, s_g) * a(s_g) \
38               * (score_fake - score_q)
39     loss_gen = stop_gradient_force_loss(X_live, force)
40     update_student_only(loss_gen)

```

Listing 5: Wrong detach and role combinations.

```

1 # WRONG 1: moving target; q_k is no longer frozen.
2 update_target_score_on_live_student_states()
3
4 # WRONG 2: old and fake are not interchangeable.
5 score_fake = old_source.predict_velocity(tau, X_live)
6
7 # WRONG 3: score networks enter generator backward.
8 loss = ((fake_score(Y) - target_score(Y)) * X_live).sum()
9

```

```

10 # WRONG 4: branch redraw inside the source ODE loop.
11 for step in source_trajectory_steps:
12     branch = torch.rand(batch_size) < alpha

```

12.3 Endpoint and TDM data flow

Endpoint MoD-DMD needs target source states only at $\tau = 1$, plus live student endpoints. MoD-TDM needs source and live-student states at sampled τ_i . A practical first version can store selected source states/callbacks and train the target score offline within each outer iteration. The generator may begin with one randomly selected, rank-broadcast segment and a one-step truncated Jacobian, clearly labeled as an approximation to (22).

Current `XTrajectoryDMTrainer` subclasses—`XTDMTrainer` and `XOPDDMTrainer`—have an additional v1 mismatch worth logging explicitly. Their generator query uses the selected segment band, but the shared fake update trains on a full-rollout endpoint with auxiliary noise sampled globally over $[0, 1]$. A strict segment-conditioned implementation should verify that fake training estimates the same $p_{\theta,u}^{(i)}$ queried by the generator, or document the endpoint-trained fake as a surrogate.

12.4 Configuration concepts, not invented accepted fields

A future configuration design needs concepts for:

- algorithm mode (endpoint MoD-DMD versus multi-time MoD-TDM);
- outer mixture weight or schedule, and old-refresh cadence;
- target-score architecture, optimizer, training steps, and freeze boundary;
- fake-score optimizer and fake-to-generator update ratio;
- τ grid or sampling law, auxiliary s law, and exact importance weights;
- sampler-gradient mode (full BPTT, checkpointed BPTT, or one-step truncation);
- event reduction, optional self-normalization, and optional Pseudo-Huber metric.

These are design requirements, not claims about existing Pydantic field names. Exact fields must be introduced through the normal training-argument synchronization and config-validation process.

Existing fields that provide nearby, but not sufficient, mechanics include:

- `xopd.target_mode`: `marginal_cfm` and `marginal_cfm_alpha`;
- `dmd.fake_ratio` and `tdm.fake_ratio`;
- `tdm.match_policy`: `random_segment`;
- `tdm.loss_metric`: `pseudo_huber`.

None currently activates a mixture-target score.

12.5 Diagnostics

At minimum, log:

- realized old/teacher branch fractions by rank and globally;
- target DSM train/holdout losses by (τ, s) and source branch;
- fake DSM loss, fake age, and fake-to-generator update ratio;
- $\|\mathbf{s}_{\text{fake}} - \mathbf{s}_q\|$ and denoiser residual quantiles by (τ, s) ;

- generator force norm before/after normalization, gradient norm, and clipping rate;
- selected segment histogram and exact w_i/π_i weights;
- old-teacher, current-target, and current-teacher sample metrics;
- target refresh number, inner step, and whether the Jacobian was full or truncated.

12.6 Distributed and mixed-precision invariants

1. **Rank-synchronous source call order.** Branch routing must make every rank enter old and teacher collectives in the same order. Synchronize branch geometry or execute both branch calls with collective-safe padding.
2. **Identical branch labels across ranks.** The Bernoulli RNG key must exclude rank and use shared values such as seed, epoch, and batch index, as in `draw_marginal_cfm_branches`. Otherwise subset shapes and teacher collectives can diverge across ranks.
3. **Broadcast the segment index.** A random TDM segment changes the number of prefix transformer calls; draw on one rank and broadcast before rollout.
4. **Current teacher-real swap.** The shipped FLUX.2-Klein `use_teacher_transformer` context swaps the whole module, bypasses the wrapped student for the no-grad teacher forward, and already disables/restores the autocast cache.
5. **Future copied-weight old snapshots.** Any no-grad loop using either of these copied-weight swap contexts,

```
adapter.use_named_parameters(name)
EMAModuleWrapper.use_ema_parameters(),
```

must disable the autocast weight cache for the full swap loop and restore its prior value afterward.

6. **DDP bypass for copied weights.** A no-grad named-parameter weight swap must temporarily route the transformer component through its unwrapped module. DDP parameter buckets can otherwise expose stale weights after `.data.copy_()` swaps.
7. **Training exception.** The gradient-enabled student pass remains through the prepared DDP/DeepSpeed wrapper so gradients synchronize correctly.
8. **Manual fake optimizer.** If fake parameters are added after `accelerator.prepare`, gradients must be explicitly all-reduced and replicas initialized identically, as in the current implementations.

12.7 Exact mapping to current files and the gap

current file / symbol	what it does now	required MoD change
<code>trainers/xopd/trainer.py</code> : <code>XOPDTrainer</code> <code>marginal_cfm</code>	A4 branch-fixed old/teacher source rollout and source-velocity MSE	reuse branch routing/call order for target-score data; add the target-score phase and an explicit freeze boundary; current $B = 0$ is the rollout-time student, not a separately materialized snapshot
<code>trainers/xopd/dmd_trainer.py</code> : <code>XDMDTrainer</code>	endpoint DMD; <code>real</code> =teacher, online <code>fake</code> =student score; consistency-style no-grad prefix plus one differentiable prediction	replace teacher-only real score by a learned frozen-per-outer $q_{k,1,s}$ score and add rollout-old snapshot/refresh
<code>trainers/xopd/traj_dm_trainer.py</code> : <code>XTDMTrainer</code>	random rank-broadcast segment; no-grad prefix plus one differentiable Euler step; teacher real score; detached live state re-noised as a clean target	learn $q_{k,\tau_i,s}$, preserve two clocks, use exact random-time weights, and label one-step Jacobian as truncated
<code>trainers/xopd/traj_dm_trainer.py</code> : <code>XOPDDMTrainer</code>	same random-segment score-difference recipe on the OPD <code>num_inference_steps</code> grid	the same mixture-target changes as XTDM; keep its grid distinction explicit
<code>trainers/xopd/traj_dm.py</code>	RF corruption, clean-estimate conversion, self-normalized force, stop-grad/Pseudo-Huber helpers, signed Euler update	retain useful mechanics but add mathematically explicit $a_s, b_s, \lambda(s)$ weighting and target-mixture interfaces

Current `XDMDTrainer` computes teacher and fake predictions at an auxiliary-noised live endpoint, then uses a self-normalized clean-estimate residual. Current `XTDMTrainer` samples one segment, executes a no-grad prefix, differentiates one Euler step, and uses the teacher as real. Neither constructs

$$(1 - \alpha)\rho_{\theta_k, \tau, s} + \alpha\rho_{T, \tau, s}.$$

Current TDM v1 does not compute the exact gradient of (22). Uniform random-segment sampling is unbiased up to an overall constant for equal segment weights, but the one-step truncated Jacobian, self-normalization, and optional Pseudo-Huber metric produce a semi-gradient rather than exact full BPTT.

13 Compact comparison

method	probability target	target signal	generator Jacobian	status / defining distinction
A4 CFM	$q_{k,\tau}$ marginal path	sampled source ODE velocity	$\partial_\theta v_\theta$ at sampled state	implemented as XOPD marginal.cfm ; density mixture without DMD fake score
ordinary XDMD	teacher endpoint density	teacher score minus online fake score	one differentiable consistency prediction	implemented; teacher is real, no old/teacher mixture
proposed MoD-DMD	$q_{k,1}$ endpoint mixture	frozen mixture score minus online fake score	endpoint sampler J_θ	new outer-target algorithm; not implemented
ordinary XTDM	teacher-real score surrogate on selected student states	teacher score minus online fake score	v1 no-grad prefix + one Euler step	implemented random-segment truncated semi-gradient, not full sum
proposed MoD-TDM	q_{k,τ_i} multi-marginals, native $q_{k,u}$, or symmetric $\tilde{q}_{k,u}^{(i)}$ segments	frozen mixture score minus online fake score	exact BPTT or declared truncation	new multi-time/segment smoothed reverse KL; not implemented

14 Limitations and recommended implementation order

14.1 Limitations

- A learned target-mixture score adds memory, compute, and an outer fitting phase.
- Teacher source trajectories remain expensive; branch sampling reduces expected cost but does not eliminate teacher rollout.
- Low old/teacher overlap weakens the first-step teacher force under reverse KL.
- Auxiliary smoothing trades overlap and score learnability against fine-detail fidelity.
- Fake-score lag can destabilize the generator; target and fake errors enter the same force.
- Finite-capacity reverse KL may mode-select instead of representing the convex mixture.
- Full multi-time BPTT can be prohibitively expensive; truncation introduces bias.
- Cross-VAE settings need an explicit common state/noise space before any mixture density is meaningful.
- Matching marginal families does not identify or match a deterministic path law.

14.2 Recommended implementation order

1. Build a CPU/test-only mathematical harness for Gaussian mixtures that verifies (8), (13), (16), and (21) by finite differences.
2. Implement endpoint target-mixture DSM data using the existing A4 branch-fixed source routing; keep the target network separate and frozen per outer loop.
3. Replace XDMD’s teacher-only real score behind a new explicitly named experimental mode; preserve ordinary XDMD unchanged.
4. Add detach, update-ratio, distributed call-order, autocast-cache, and DDP-bypass tests.
5. Validate endpoint MoD-DMD on a same-VAE, small-resolution setup with exact unnormalized weighting before introducing self-normalization or Pseudo-Huber.

6. Extend target-score conditioning to trajectory time τ and verify random-time importance weights.
7. Add MoD-TDM first with current one-step truncation, naming and logging it as such; compare against short-horizon full BPTT on tiny models.
8. Only then study dynamic α_k , three-way anchors, cross-VAE transport, or classifier routing.

15 Internal documents and final checklist

The closest internal references are:

- docs/xopd/arm_a4_marginal_mixture_cfm_tutorial.pdf for A4;
- docs/xopd/dmd_opd_gradient_relation.pdf for OPD versus endpoint DMD;
- docs/xopd/tdm_opd_dm_gradient_relation.tex for current TDM Jacobians;
- docs/xopd/popd_exact_sum_gate_saturation.pdf for transition-gate behavior;
- docs/xopd/flow_direct_opd_small_rl_delta_transfer.pdf for direct OPD context.

Minimum mathematically faithful MoD implementation

- ✓ Old means frozen θ_k , not current and not fixed pretrained base.
- ✓ Source branch is sampled once per complete trajectory.
- ✓ τ and s are stored and conditioned separately.
- ✓ Target score represents $q_{k,\tau,s}$, not teacher alone.
- ✓ Fake score trains on detached live-student states.
- ✓ Generator sign is $\mathbf{s}_{\text{fake}} - \mathbf{s}_q$; descent moves toward $\mathbf{s}_q - \mathbf{s}_{\text{fake}}$.
- ✓ Denoiser form includes a_s^2/b_s^2 , or changed weighting is disclosed.
- ✓ Target and fake scores are detached in generator backward.
- ✓ Random-time weights and scheduler orientation are explicit.
- ✓ Truncated Jacobians are not described as exact BPTT.
- ✓ Claims concern smoothed marginals, not deterministic path-space KL.
- ✓ Distributed named-parameter swaps obey call-order, autocast-cache, and DDP invariants.

16 Conclusion

MoD-DMD and MoD-TDM combine two ideas without conflating their mathematical objects. A4 supplies a frozen old/teacher *density target* in the outer-loop mathematical sense; shipped `marginal_cfm` instead uses the rollout-time student on the $B = 0$ branch and caches its velocity target. DMD/TDM supply a score-difference *distribution-matching update*. Gaussian smoothing makes the target score learnable, branch-fixed source sampling avoids explicit density estimation, and the online fake score provides the entropy side of reverse KL. The exact endpoint force is

$$a_s J_\theta^\top (\mathbf{s}_{\text{fake}} - \mathbf{s}_q),$$

and the first inner step is gated by the smoothed posterior responsibility. Multi-time matching may use independently smoothed ODE marginals or the paper-style segment-conditioned laws in (25); these are distinct unless their kernels and marginal-consistency identities agree. In either case, sampler sensitivity—exact or explicitly truncated—carries the score force to parameters.

The practical boundary is equally important: current XDMD and XTDM are teacher-real methods. Turning them into MoD methods requires a frozen rollout-old source, a separately trained old/teacher target-mixture score, an outer refresh protocol, and new diagnostics and invariants. Changing a label from “teacher” to “mixture” or averaging source velocities would not implement the proposal.

References

- [1] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2210.02747>.
- [2] Weijia Luo, Zhiwei Hu, Yijun Zhang, et al. Diff-Instruct: A Universal Approach for Transferring Knowledge From Pre-trained Diffusion Models. *Advances in Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2305.18455>.
- [3] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T. Freeman, and Taesung Park. One-step Diffusion with Distribution Matching Distillation. *CVPR*, 2024. <https://arxiv.org/abs/2311.18828>.
- [4] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Frédo Durand, and William T. Freeman. Improved Distribution Matching Distillation for Fast Image Synthesis. *NeurIPS*, 2024. <https://arxiv.org/abs/2405.14867>.
- [5] Weijian Luo et al. Learning Few-Step Diffusion Models by Trajectory Distribution Matching. *ICCV*, 2025. <https://arxiv.org/abs/2503.06674>.
- [6] Bradley Efron. Tweedie’s Formula and Selection Bias. *Journal of the American Statistical Association*, 2011. <https://doi.org/10.1198/jasa.2011.tm11181>.